```
function sig2_est =
ps_vce(Npsc,Nifgs,max_arc_length,model,Btemp,Bdop,std_param,breakpoint,breakpoin
t2)

% Function to estimate a variance factor for each interferogram
%
% Input:  - Npsc              number of psc
%         - Nifgs             number of interferograms
%         - max_arc_length    maximum arc length
%         - model             vector with model indices to be
%                             evaluated (see help ps_model_definitions
%                             for more information)
%         - Btemp             temporal baselines [year]
%         - Bdop              Doppler baselines [Hz]
%         - std_param         standard deviations for pseudo-
%                             observations
%         - breakpoint        breakpoint in case of double
%                             linear model
%         - breakpoint2       second breakpoint, should be larger
%                             than first breakpoint
%
% Output: - sig2_est           variance components
%
% ----------------------------------------------------------------------
% File............: ps_vce.m
% Version & Date..: 1.7.2.16, 12-DEC-2009
% Author..........: Freek van Leijen
%                   Delft Institute of Earth Observation and Space Systems
%                   Delft University of Technology
% ----------------------------------------------------------------------
%
% This software is developed by Delft University of Technology and is
% intended for scientific use only. Applications for commercial use are
% prohibited.
%
% Copyright (c) 2004-2009 Delft University of Technology, The Netherlands
%
% Change log
%


% ----------------------------------------------------------------------
% Initialize
% ----------------------------------------------------------------------

global az_spacing r_spacing project_id ps_method ens_coh_threshold
global althyp_index althyp_index2

step1_orig = 1; % [m]
step2_orig = 0.0001; % [m], = 0.1 mm


% ----------------------------------------------------------------------
% Read data
% ----------------------------------------------------------------------

z = 1; %only for first selection

psc_fid = fopen([project_id '_psc_sel' num2str(z) '.raw'],'r'); % file with
temporary psp info
psc_data = fread(psc_fid,[2*Nifgs+7 Npsc(z)],'double')';
fclose(psc_fid);

psc_array = psc_data(:,1:2);
```

```
psc_index = find(psc_array(:,2)~=0);
psc_data = psc_data(psc_index,:);

psc_az = psc_data(:,5);
psc_r = psc_data(:,6);
psc_phase = psc_data(:,7:Nifgs+6);
psc_h2ph = psc_data(:,Nifgs+7:2*Nifgs+6);
clear psc_data

dpsc_phase = repmat(NaN,3*Npsc(z)-3,Nifgs);
dpsc_h2ph = repmat(NaN,3*Npsc(z)-3,Nifgs);
dpsc_arcs = repmat(NaN,3*Npsc(z)-3,2);
% based on theoretical maximum number of arcs for Delaunay


% ---------------------------------------------------------------------
% Triangulation by Delaunay
% ---------------------------------------------------------------------

trian = delaunay(psc_az,(r_spacing/az_spacing)*psc_r);
% to get a 'nice' triangulation
trian = sort(trian,2);
% operations to extract the unique arcs from the Delaunay triangulation
A = trian(:,1:2);
B = trian(:,2:3);
C = trian(:,1:2:3);
a = union(A,B,'rows');
dpsc_arcs_orig = union(a,C,'rows');

% select independent arcs
v = 1;
while v<size(dpsc_arcs_orig,1)
  index = dpsc_arcs_orig(v,1);
  dpsc_arcs_orig(find(dpsc_arcs_orig(v+1:end,1)==index)+v,:) = [];
  dpsc_arcs_orig(find(dpsc_arcs_orig(v+1:end,2)==index)+v,:) = [];
  index = dpsc_arcs_orig(v,2);
  dpsc_arcs_orig(find(dpsc_arcs_orig(v+1:end,1)==index)+v,:) = [];
  dpsc_arcs_orig(find(dpsc_arcs_orig(v+1:end,2)==index)+v,:) = [];
  v = v+1;
end
Narcs_orig = size(dpsc_arcs_orig,1);

%%% The number of sides can be checked theoretically by:
%%% Nsides = 3*Nps - 3 - k, where k is the number of points
%%% on the so-called convex hull boundary (outside boundary of ps set). So
%%%
%%% K = convhull(psc_az,psc_r)
%%% k = length(K)-1
%%% theo_Narcs = 3*length(psc_az)-3-k
%%%
%%% if (Narcs ~= theo_Narcs)
%%%   error('Something went wrong in the triangulation process')
%%% end
%%%
%%% However, because integer pixel cn are used, sometimes a ps lies
%%% exactly between two other ps, and 'convhull' makes numerical
%%% mistakes. 'delaunay' seems to work correct.


% ---------------------------------------------------------------------
% Select phase differences for arcs shorter than atmospheric correlation length
% ---------------------------------------------------------------------

count = 0;
```

```
for v = 1:Narcs_orig
    dist = sqrt((az_spacing*(psc_az(dpsc_arcs_orig(v,1)) - ...
                             psc_az(dpsc_arcs_orig(v,2))))^2 + ...
                (r_spacing*(psc_r(dpsc_arcs_orig(v,1))-
psc_r(dpsc_arcs_orig(v,2))))^2);

    if (dist < max_arc_length)
        count = count + 1;
        dpsc_phase(count,:) = psc_phase(dpsc_arcs_orig(v,2),:)- ...
            psc_phase(dpsc_arcs_orig(v,1),:);
        dpsc_h2ph(count,:) = (psc_h2ph(dpsc_arcs_orig(v,2),:)+ ...
                              psc_h2ph(dpsc_arcs_orig(v,1),:))/2; % take the
mean value of the arc
        dpsc_arcs(count,:) = dpsc_arcs_orig(v,:);
    end
end

Narcs_vce = count;



% ---------------------------------------------------------------------
% Remove NaN lines
% ---------------------------------------------------------------------

dpsc_phase = dpsc_phase(1:Narcs_vce,:);
dpsc_h2ph = dpsc_h2ph(1:Narcs_vce,:);
dpsc_arcs = dpsc_arcs(1:Narcs_vce,:);


% ---------------------------------------------------------------------
% Temporal unwrapping
% ---------------------------------------------------------------------

model = model(1);

switch ps_method
  case 'perio'

    [dpsc_param,dpsc_acheck,dpsc_model_info,dpsc_ens_coh] =
ps_periodogram_estimation(dpsc_h2ph,Btemp,dpsc_phase,model,std_param,step1_orig,
step2_orig);

    dpsc_varfac_best = dpsc_ens_coh;
    varfac_index = dpsc_varfac_best<ens_coh_threshold;
    dpsc_varfac_best(varfac_index) = NaN;

  case {'ils','boot'}

    [dpsc_param,dpsc_acheck,dpsc_model_info,dpsc_varfac_best,dpsc_ens_coh] =
ps_ils_and_bootstrap_estimation(dpsc_h2ph,Btemp,Bdop,dpsc_phase,model,std_param,
[],breakpoint,breakpoint2);

    if isempty(find(~isnan(dpsc_model_info(:,1))));
        error('Something went wrong in the estimation of the network. Most
likely the stochastic model used is incorrect.');
    end

  otherwise

    error('Please select a valid ps method: ''period'', ''ils'' or ''boot''');

end
```

```
NaN_index = find(isnan(dpsc_varfac_best));
dpsc_param(NaN_index,:) = [];
dpsc_acheck(NaN_index,:) = [];
dpsc_model_info(NaN_index,:) = [];
dpsc_varfac_best(NaN_index) = [];
dpsc_ens_coh(NaN_index) = [];
dpsc_h2ph(NaN_index,:) = [];
dpsc_phase(NaN_index,:) = [];
Narcs_vce = length(dpsc_varfac_best);


% ------------------------------------------------------------------------
% Variance Component Estimation (VCE)
% ------------------------------------------------------------------------

if ~isempty(breakpoint)
    althyp_index = breakpoint;
else
    althyp_index = 2; % for proper flow of models
end

if ~isempty(breakpoint2)
    althyp_index2 = breakpoint2;
else
    althyp_index2 = NaN;
end

mean_h2ph = mean(dpsc_h2ph,1)';
B1Qy2 =
ps_model_definitions('model',model,Nifgs,mean_h2ph,Btemp,Bdop,std_param);
B = B1Qy2(1:Nifgs,:);
dpsc_unwrap = 2*pi*dpsc_acheck+dpsc_phase;

if model == 1 % no master atmosphere estimated, hence master
              % atmosphere stochastic
    Nsig = Nifgs+1;
    sig0 = [(pi*15/180)^2; repmat((pi*20/180)^2,Nifgs,1)];

    Qy1 = zeros(Nifgs,Nifgs,Nsig);
    Qy1(:,:,1) = repmat(2,Nifgs,Nifgs);
    for v = 1:Nifgs
        Qy1(v,v,v+1) = 2;
    end

    Qy = repmat(2*sig0(1),Nifgs,Nifgs);
    for v = 1:Nifgs
        Qy(v,v) = Qy(v,v)+2*sig0(v+1);
    end

else %
    Nsig = Nifgs;
    sig0 = [repmat((pi*30/180)^2,Nifgs,1)];

    Qy1 = zeros(Nifgs,Nifgs,Nsig);
    for v = 1:Nifgs
        Qy1(v,v,v) = 2;
    end

    Qy = zeros(Nifgs,Nifgs);
    for v = 1:Nifgs
        Qy(v,v) = Qy(v,v)+2*sig0(v);
    end

end
```

```
Qyinv = inv(Qy);

l = repmat(NaN,Nsig,1);
N = repmat(NaN,Nsig,Nsig);

Pao=eye(Nifgs)-B*inv(B'*Qyinv*B)*B'*Qyinv;
QP = Qyinv*Pao;

QPQy1QP = repmat(NaN,[Nifgs Nifgs Nsig]);
for k=1:Nsig
    QPQy1QP(:,:,k) = QP*Qy1(:,:,k)*QP;
    for j=1:Nsig
        N(k,j)=trace(QPQy1QP(:,:,k)*Qy1(:,:,j));
    end
end
Ninv=inv(N);

sig2 = repmat(NaN,Nsig,Narcs_vce);
for v = 1:Narcs_vce
    y = dpsc_unwrap(v,:)';
    for k=1:Nsig
        l(k,1)=y'*QPQy1QP(:,:,k)*y;
    end
    sig2(:,v)=Ninv*l;
end
sig2_est = mean(sig2,2);

index = find(sig2_est<(pi*10/180)^2); % avoid small and negative values
sig2_est(index) = (pi*10/180)^2;

if model ~= 1
    sig2_est = [0;sig2_est]; %add a zero for master atmosphere variance
end
```